I/O ORGANIZATION

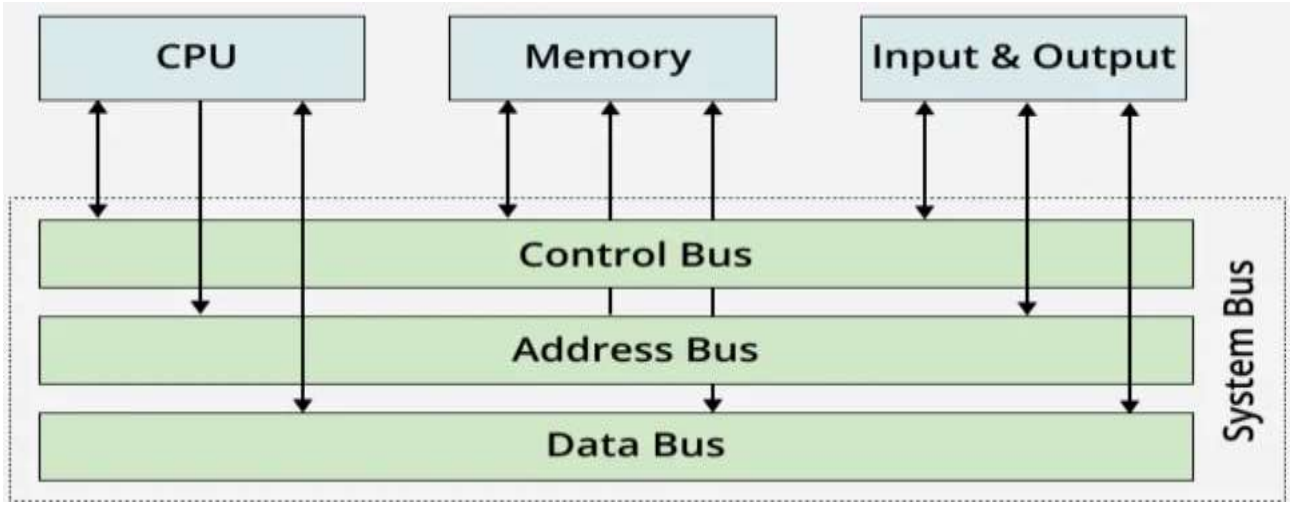
UNIT-IV

I/O ORGANIZATION

- Bus control
- Serial I/O (study of Asynchronous and synchronous modes, USART & UART)
- Parallel Data transfer
- (Program controlled: Asynchronous, synchronous & Interrupt driven modes, DMA mode, interrupt controller and DMA controller)
- Buses Device subsystem
- External storage system
- RAID architecture

Bus control

- A computer bus is a communication system used to transfer data between components within a computer or between different computers. It plays an important role in minimising the number of connections needed by centralising communication over shared pathways.
- It consists of physical connections like wires, circuits, or cables.
- Components like the CPU, memory, and input/output (I/O) devices are connected through a bus.
- It simplifies data transfer and improves efficiency.
- **Types of Buses**
- There are three main types of buses in a computer system, which are discussed below:



Bus control

- **1. Address Bus**
- A collection of wires used to identify a particular location in main memory is called the Address Bus. Or, in other words, the information used to describe the memory locations travels along the address bus.
- The address bus transports memory addresses, which the processor wants to access in order to read or write data..
- The address bus is unidirectional.
- The size of the address bus determines how many unique memory locations can be addressed.
- **2. Data Bus**
- A collection of wires through which data is transmitted from one part of a computer to another is called a Data Bus. It can be thought of as a highway on which data travels within a computer.
- The main objective of the data bus is the transfer of data between the microprocessor and input/ output devices or memory.
- The data bus transfers instructions coming from or going to the processor.
- The data bus is bidirectional because the data can flow in either direction from CPU to memory(or input/output device) or from memory to the CPU.
- The size (width) of the bus determines how much data can be transmitted at one time.

Bus control

- **3. Control Bus**
- The connections that carry control information between the CPU and other devices within the computer are called the Control Bus. The control bus transports orders and synchronization signals coming from the control unit and traveling to all other hardware components

- The main objective of the control bus is to carry all signals from the processor to other hardware devices.
- The Control bus is bidirectional because the data can flow in either direction from CPU to memory(or input/output device) or from memory to the CPU.
- it also transmits response signals from the hardware.
- Example:
- This bus is used to indicate whether the CPU is reading from memory or writing to memory.

Serial I/O (study of Asynchronous and synchronous modes, USART & VART)

- Serial IO" (Input/Output) refers to a method of transmitting data one bit at a time sequentially between devices, or to the Intel Serial IO (SIO) technology integrated into Intel hardware. Intel's SIO is a collection of drivers and hardware components, such as controllers for I2C, SPI, and UART, that manage communication between the system and its peripherals. The term can also refer to companies like Serialio.com, which develop technology for data tracking using hardware like RFID and barcode readers

Memory-CPU Bus

- Important characteristics:
  - Short: processor and memory are on the same board.
  - Synchronous: High-speed
  - Matched to the memory system access.
  - Usually proprietary.

I/O Buses

- Also called peripheral buses:
  - Longer: Devices can be meters away.
  - Asynchronous: Handshaking
    - Becomes possible to connect many types of device: some slow devices and others faster devices.
  - Follow bus standard.

Synchronous Bus

- A synchronous bus uses a clock:
  - Requests and replies can only be sent on clock ticks.
  - Fast and cheap:
    - No logic to make decisions about when to expect data.
  - Two main problems:
    - These buses cannot be long: Clock skew problem
    - Everything on the bus must work at the same clock rate:
      - Or at least the bus works at the rate of the slowest connected device
  - Typical synchronous bus: CPU-memory bus

Asynchronous Bus

- An asynchronous bus does not use a clock:
  - Components connected on the bus use some "hand-shaking" protocol: Req..Ack messages.
  - Easier to accommodate a wide variety of devices.
  - Makes it possible to lengthen the bus without worrying about clock skew.
  - Slower than a synchronous bus:
    - Hand-shaking incurs overhead.
  - Typical asynchronous bus: I/O bus

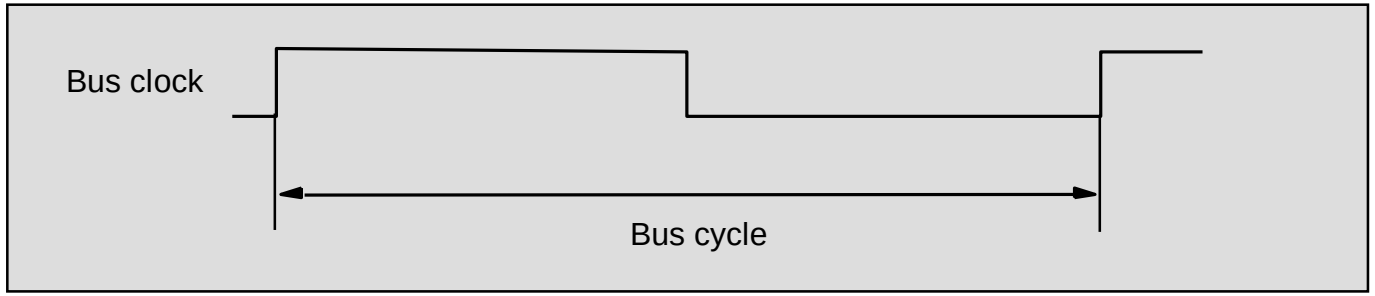
Bus Masters/Slaves

- Master: A device that can initiate a transaction on the bus:
  - A read or a write
  - The CPU is always a master.
- Slave: Activated by a transaction.
- Single masters are simpler but slower.
- Multiple bus masters:
  - More than one CPU, or when I/O devices can initiate bus transactions too.
  - Requires an arbitration scheme.

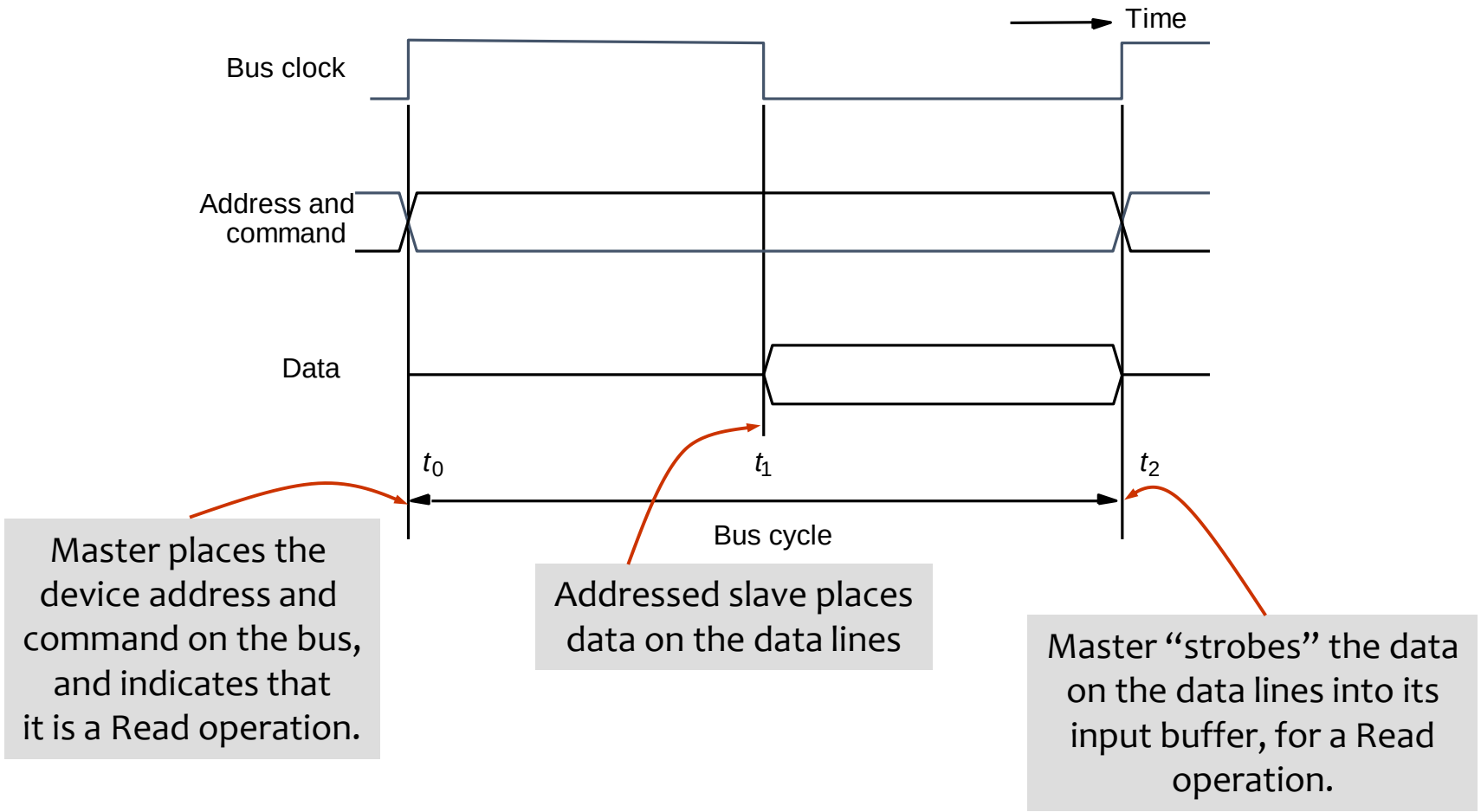
Buses (contd..)

- Bus lines may be grouped into three types:
  - Data
  - Address
  - Control
- Control signals specify:
  - Whether it is a read or a write operation.
  - Required size of the data, when several operand sizes (byte, word, long word) are possible.
  - Timing information to indicate when the processor and I/O devices may place data or receive data from the bus.
- Schemes for timing of data transfers over a bus can be classified into:
  - Synchronous,
  - Asynchronous.

Synchronous bus



Synchronous bus (contd..)



- *In case of a Write operation, the master places the data on the bus along with the address and commands at time t_o.*
- *The slave strobes the data into its input buffer at time t_2.*

Synchronous bus (contd..)

- Once the master places the device address and command on the bus, it takes time for this information to propagate to the devices:
  - This time depends on the physical and electrical characteristics of the bus.
- Also, all the devices have to be given enough time to decode the address and control signals, so that the addressed slave can place data on the bus.
- Width of the pulse t_1 - t_0 depends on:
  - Maximum propagation delay between two devices connected to the bus.
  - Time taken by all the devices to decode the address and control signals, so that the addressed slave can respond at time t_1.

Synchronous bus (contd..)

- At the end of the clock cycle, at time t_2, the master strobes the data on the data lines into its input buffer if it's a Read operation.
  - "Strobe" means to capture the values of the data and store them into a buffer.
- When data are to be loaded into a storage buffer register, the data should be available for a period longer than the setup time of the device.
- Width of the pulse t_2 - t_1 should be longer than:
  - Maximum propagation time of the bus plus
  - Set up time of the input buffer register of the master.

Synchronous bus (contd..)

- Data transfer has to be completed within one clock cycle.
  - Clock period t2 - t0 must be such that the longest propagation delay on the bus and the slowest device interface must be accommodated.
  - Forces all the devices to operate at the speed of the slowest device.
- Processor just assumes that the data are available at t2 in case of a Read operation, or are read by the device in case of a Write operation.
  - What if the device is actually failed, and never really responded?

Synchronous bus (contd..)

- Most buses have control signals to represent a response from the slave.
- Control signals serve two purposes:
  - Inform the master that the slave has recognized the address, and is ready to participate in a data transfer operation.
  - Enable to adjust the duration of the data transfer operation based on the speed of the participating slaves.
- High-frequency bus clock is used:
  - Data transfer spans several clock cycles instead of just one clock cycle as in the earlier case.

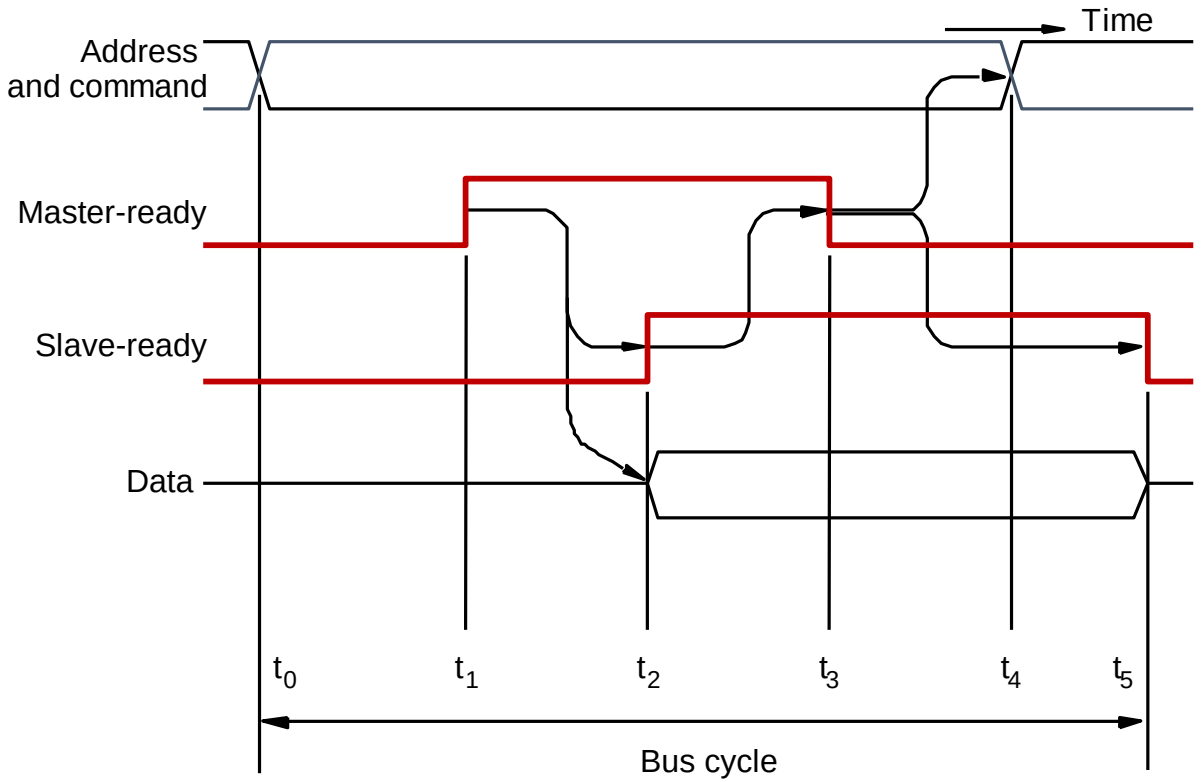
Asynchronous bus

- Data transfers on the bus is controlled by a handshake between the master and the slave.
- Common clock in the synchronous bus case is replaced by two timing control lines:
  - Master-ready,
  - Slave-ready.
- Master-ready signal is asserted by the master to indicate to the slave that it is ready to participate in a data transfer.
- Slave-ready signal is asserted by the slave in response to the master-ready from the master, and it indicates to the master that the slave is ready to participate in a data transfer.

Asynchronous bus (contd..)

- Data transfer using the handshake protocol:
  - Master places the address and command information on the bus.
  - Asserts the Master-ready signal to indicate to the slaves that the address and command information has been placed on the bus.
  - All devices on the bus decode the address.
  - Address slave performs the required operation, and informs the processor it has done so by asserting the Slave-ready signal.
  - Master removes all the signals from the bus, once Slave-ready is asserted.
  - If the operation is a Read operation, Master also strobes the data into its input buffer.

Asynchronous bus (contd..)



t_o - Master places the address and command information on the bus.
t_1 - Master asserts the Master-ready signal. Master-ready signal is asserted at t_1 instead of t_o
t_2 - Addressed slave places the data on the bus and asserts the Slave-ready signal.
t_3 - Slave-ready signal arrives at the master.
t_4 - Master removes the address and command information.
t_5 - Slave receives the transition of the Master-ready signal from 1 to 0. It removes the data and the Slave-ready signal from the bus.

Asynchronous vs. Synchronous bus

- Advantages of asynchronous bus:
  - Eliminates the need for synchronization between the sender and the receiver.
  - Can accommodate varying delays automatically, using the Slave-ready signal.
- Disadvantages of asynchronous bus:
  - Data transfer rate with full handshake is limited by two-round trip delays.
  - Data transfers using a synchronous bus involves only one round trip delay, and hence a synchronous bus can achieve faster rates.

Introduction: UART vs USART

UART stands for **Universal Asynchronous Receiver Transmitter**.

USART stands for **Universal Synchronous Asynchronous Receiver Transmitter**.

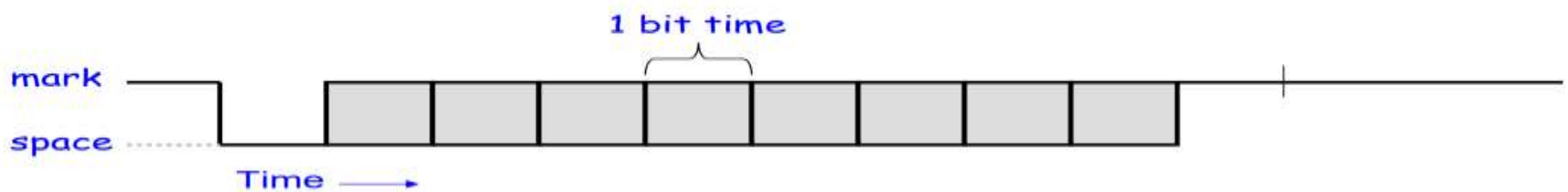
They are basically just a piece of hardware that converts parallel data into serial data.

Why use a UART?

- A UART may be used when:
  - High speed is not required
  - A cheap communication line between **two** devices is required
- Asynchronous serial communication is very cheap
  - Requires a transmitter and/or receiver
  - Single wire for each direction (plus ground wire)
  - Relatively simple hardware
  - Asynchronous because the
- PC devices such as mice and modems used to often be asynchronous serial devices

UART Character Transmission

- Below is a timing diagram for the transmission of a single byte
- Uses a single wire for transmission
- Each block represents a bit that can be a **mark** (logic '1', high) or **space** (logic '0', low)



Universal Synchronous Asynchronous Receiver Transmitter

- Universal Synchronous Asynchronous Receiver Transmitter
- Can be synchronous or asynchronous
- Can receive and transmit
- Full duplex asynchronous operation

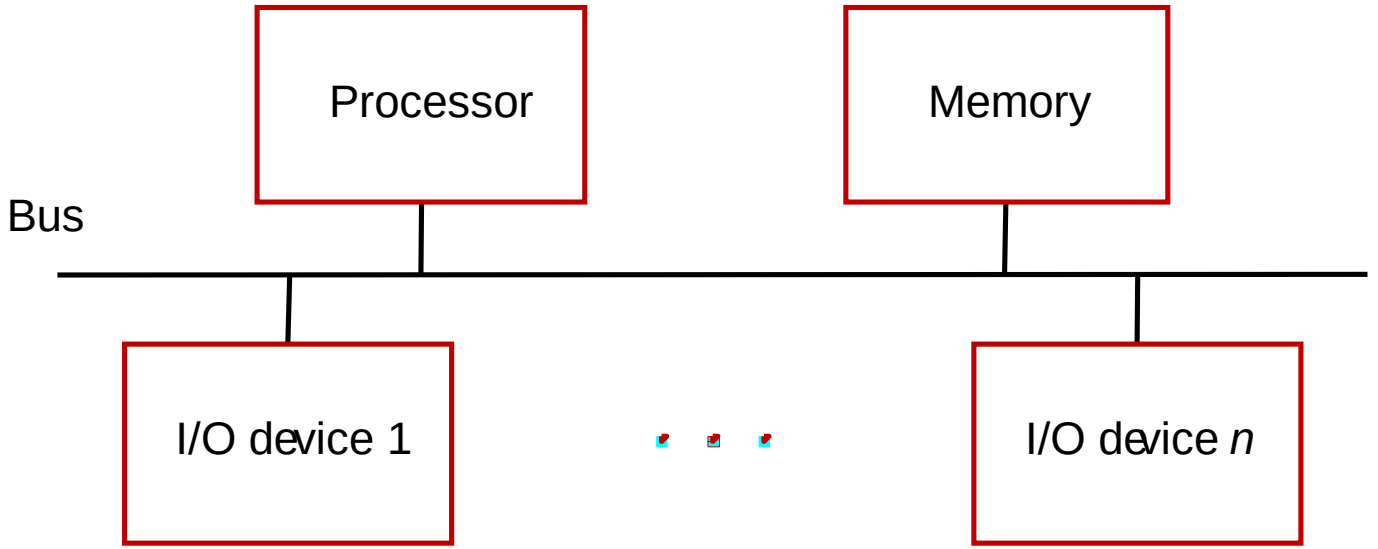
Most common use : RS-232 communications to a PC serial port

Difference Between USART and UART

USART	UART
In USART, half duplex mode is used.	While in UART, full duplex mode is used.
The speed of USART is more than the speed of UART.	While the speed of UART is comparatively less.
USART uses both data signals and clock for its functioning.	While UART entails data signals only for its functioning.
In USART, data is transmitted in the form of blocks.	While in UART, data is transmitted in the form of bytes(one byte at a time).
USART can do its function like UART.	Whereas UART can't do its function like USART.
USART is more complex than UART in terms of complexity.	While UART is simple in terms of complexity.
In USART, receiver doesn't have to know the baud-pace of the transmitter as it is gotten from the information line gave by the master and the clock signal.	While there is no approaching clock signal that is related with the information, so the recipient has to know baud-pace of the transmitter before the inception of gathering.
In the USART, data is transmitted at a definite rate.	While in UART, data can be transmitted at a variable speed.

- Program-controlled" refers to a system where a set of instructions stored in memory directs the operation of the hardware, enabling automatic decision-making and altering the sequence of execution based on conditions or events.
- This control is achieved through program control instructions that modify the program counter, changing the next instruction to be executed and allowing for complex logic like loops, conditional branches, and subroutines.

Accessing I/O devices

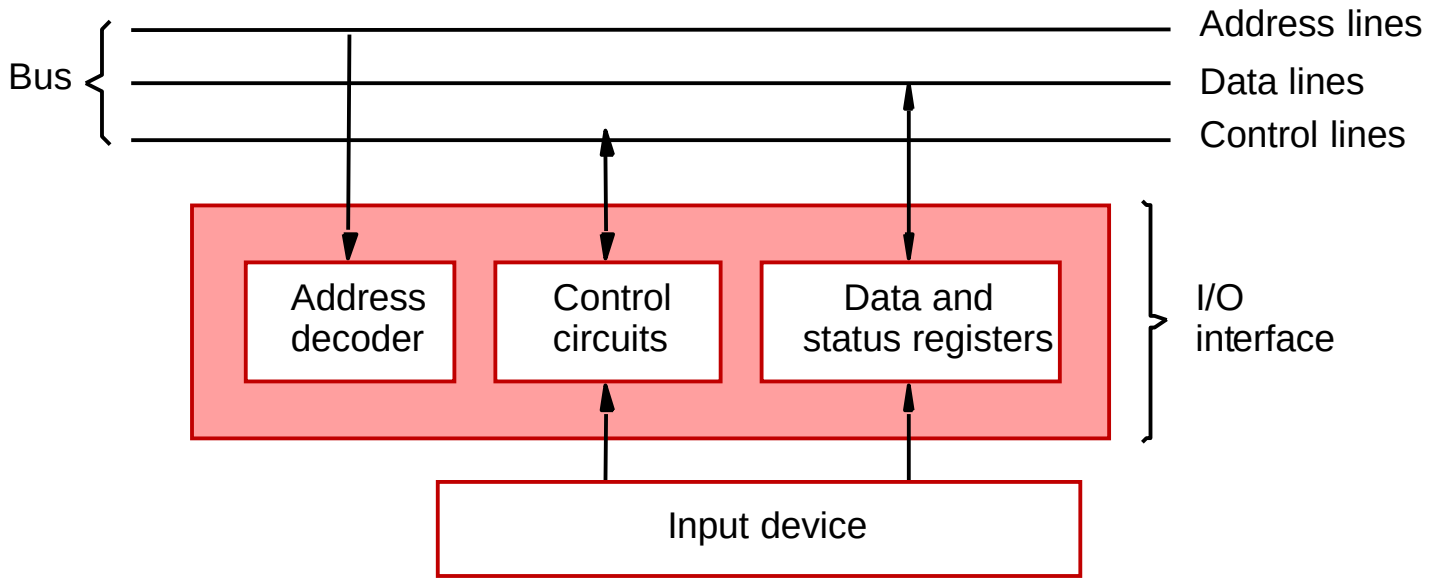


- *Multiple I/O devices may be connected to the processor and the memory via a bus.*
- *Bus consists of three sets of lines to carry address, data and control signals.*
- *Each I/O device is assigned an unique address.*
- *To access an I/O device, the processor places the address on the address lines.*
- *The device recognizes the address, and responds to the control signals.*

Accessing I/O devices (contd..)

- I/O devices and the memory may share the same address space:
  - Memory-mapped I/O.
  - Any machine instruction that can access memory can be used to transfer data to or from an I/O device.
  - Simpler software.
- I/O devices and the memory may have different address spaces:
  - Special instructions to transfer data to and from I/O devices.
  - I/O devices may have to deal with fewer address lines.
  - I/O address lines need not be physically separate from memory address lines.
  - In fact, address lines may be shared between I/O devices and memory, with a control signal to indicate whether it is a memory address or an I/O address.

Accessing I/O devices (contd..)



- *I/O device is connected to the bus using an I/O interface circuit which has:*
  - *Address decoder, control circuit, and data and status registers.*
- *Address decoder decodes the address placed on the address lines thus enabling the device to recognize its address.*
- *Data register holds the data being transferred to or from the processor.*
- *Status register holds information necessary for the operation of the I/O device.*
- *Data and status registers are connected to the data lines, and have unique addresses.*
- *I/O interface* ***circuit coordinates I/O transfers.***

Accessing I/O devices (contd..)

- Recall that the rate of transfer to and from I/O devices is slower than the speed of the processor. This creates the need for mechanisms to synchronize data transfers between them.
- Program-controlled I/O:
  - Processor repeatedly monitors a status flag to achieve the necessary synchronization.
  - Processor polls the I/O device.
- Two other mechanisms used for synchronizing data transfers between the processor and memory:
  - Interrupts.
  - Direct Memory Access.

Interrupt driven modes

- **Interrupt-driven** modes, most commonly referring to interrupt-driven I/O, are a method for handling input/output operations where the I/O device signals the processor via an interrupt when it's ready for a data transfer, allowing the CPU to perform other tasks in the meantime.
- After receiving the interrupt, the CPU suspends its current activity, executes an interrupt service routine (ISR) to transfer data, and then resumes its original task, making efficient use of CPU time compared to programmed I/O.
- **I/O Request:** The processor issues a command to the I/O module to start an operation (e.g., read data from a device).
- **CPU Multitasking:** The CPU does not wait for the I/O device to complete its task. Instead, it proceeds to execute other instructions or perform other computations.
- **Interrupt Signal:** When the I/O device has the data ready, it sends an interrupt signal to the CPU, indicating its readiness.
- **Context Switch:** Upon receiving the interrupt signal, the CPU completes the current instruction, saves its current state (context) onto a stack, and loads the address of the interrupt service routine (ISR) into the program counter.
- **Interrupt Service Routine:** The ISR is executed, which handles the data transfer between the I/O device and memory.

DMA mode

- Direct Memory Access is a method that allows hardware peripherals to transfer data directly to or from main memory, bypassing the CPU.
- This process is managed by a DMA controller and requires the system bus to operate. Different DMA transfer modes determine how the DMA controller uses the system bus and interacts with the CPU.

Burst Mode

- In burst mode, the DMA controller takes control of the system bus and continues to transfer data in blocks until the entire transfer is complete.
- **CPU impact:** During a burst transfer, the CPU has to wait and cannot use the bus until the data transfer is finished.
- **Best for:** Large data transfers where high throughput is required and interruptions to the CPU are acceptable.

DMA mode

Cycle Stealing Mode

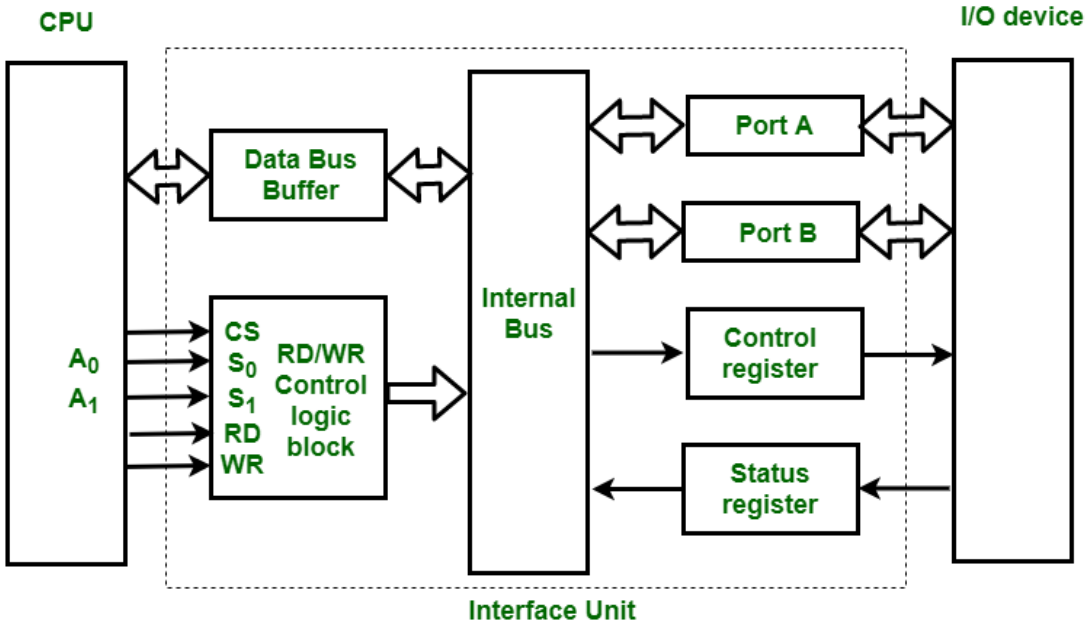
- The DMA controller requests control of the bus for short periods, performing a single-byte (or single-word) transfer, then returns the bus to the CPU.
- **CPU impact:** The CPU experiences minimal delays because it can regain control of the bus after each small transfer.
- **Best for:** Slow I/O devices where data is prepared in smaller chunks, allowing the CPU to perform other tasks without long waits.

Transparent Mode

- The DMA controller transfers data only during periods when the CPU is not actively using the system bus (i.e., during its "idle" cycles).
- **CPU impact:** This mode minimizes impact on CPU performance because the CPU's tasks are not interrupted.
- **Best for:** Situations where the CPU is performing complex operations that don't require bus access, allowing for efficient background data transfers.

Interrupt controller and DMA controller

- The method that is used to transfer information between internal storage and external I/O devices is known as I/O interface.
- The CPU is interfaced using special communication links by the peripherals connected to any computer system. These communication links are used to resolve the differences between the CPU and the peripheral.



Interrupt controller and DMA controller

Mode of Transfer

- The binary information that is received from an external device is usually stored in the memory unit. CPU merely processes the information but the source and target is always the memory unit. Data transfer between CPU and the I/O devices may be done in different modes.
- **Programmed I/O.**
- **Interrupt- initiated I/O.**
- **Direct memory access(DMA).**
- **Programmed I/O**
- In this, each data item transfer is initiated by an instruction in the program
- Usually, the transfer is from a CPU register to memory. In this case it requires constant monitoring by the CPU of the peripheral devices.
- In this case, the I/O device does not have direct access to the memory unit. A transfer from I/O device to memory requires the execution of several instructions by the CPU
- Including an input instruction to transfer the data from device to the CPU and store instruction to transfer the data from CPU to memory.

Interrupt controller and DMA controller

Interrupt-Initiated I/O

Since in the above case we saw the CPU is kept busy unnecessarily. This situation can very well be avoided by using an interrupt driven method for data transfer. By using interrupt facility and special commands to inform the interface to issue an interrupt request signal whenever data is available from any device.

- In the meantime, the CPU can proceed with any other program execution.
- The interface meanwhile keeps monitoring the device.
- Whenever it is determined that the device is ready for data transfer it initiates an interrupt request signal to the computer.
- Upon detection of an external interrupt signal the CPU stops momentarily the task that it was already performing, branches to the service program to process the I/O transfer and then return to the task it was originally performing.
- The I/O transfer rate is limited by the speed with which the processor can test and service a device.
- The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer.

Interrupt controller and DMA controller

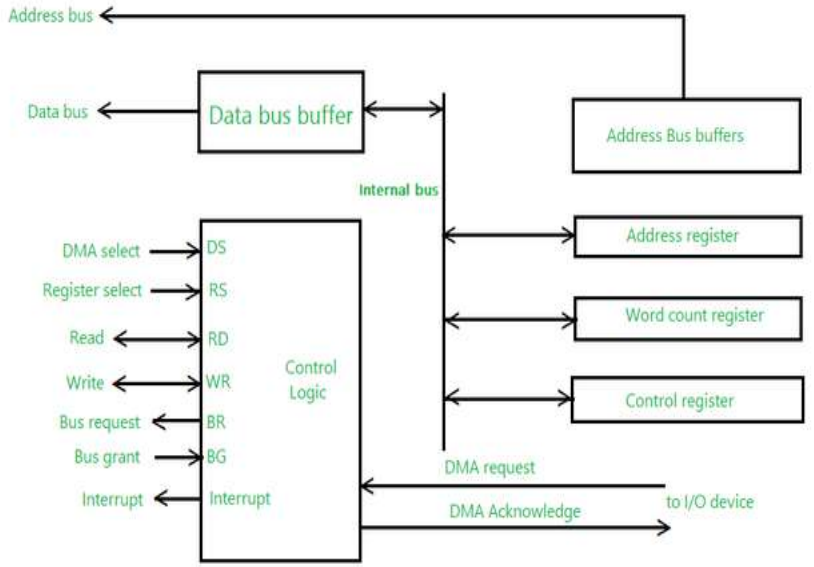
Direct Memory Access

The data transfer between a fast storage media such as magnetic disk and memory unit is limited by the speed of the CPU. Thus we can allow the peripherals directly communicate with each other using the memory buses, removing the intervention of the CPU. This type of data transfer technique is known as DMA or direct memory access.

During DMA the CPU is idle and it has no control over the memory buses.

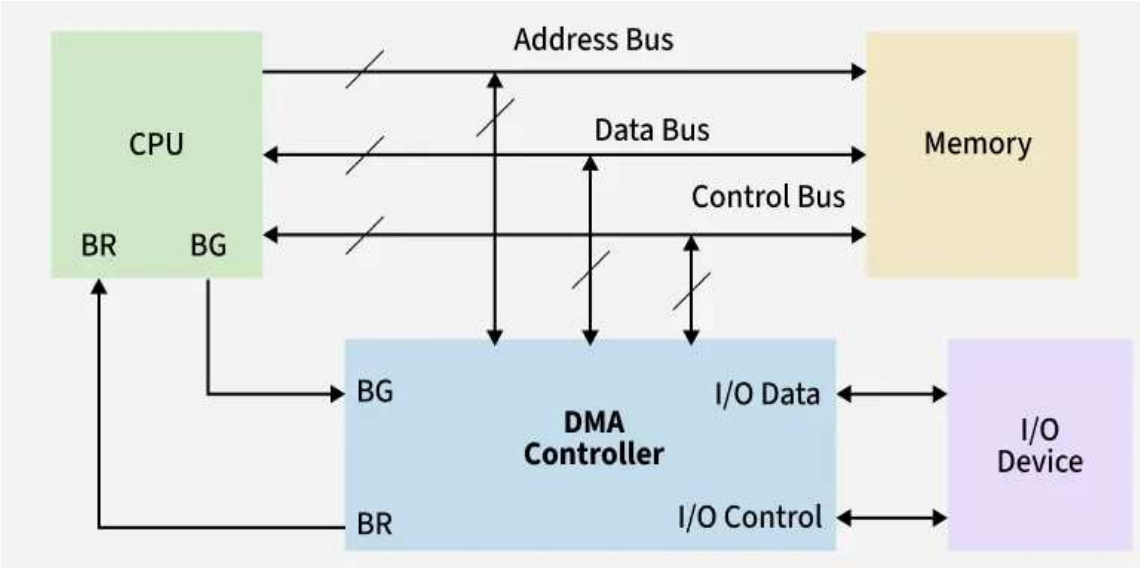
The DMA controller takes over the buses to manage the transfer directly between the I/O devices and the memory unit. Bus grant request time.

Transfer the entire block of data at transfer rate of device because the device is usually slow than the speed at which the data can be transferred to CPU. Release the control of the bus back to CPU So,



DMA controller

DMA Controller is a type of control unit that works as an interface for the data bus and the I/O Devices. As mentioned, DMA Controller has the work of transferring the data without the intervention of the processors, processors can control the data transfer. DMA Controller also contains an address unit, which generates the address and selects an I/O device for the transfer of data. Below is the block diagram of the DMA Controller.



DMA controller

Types of Direct Memory Access (DMA)

There are four popular types of DMA.

Single-Ended DMA: In this type, the DMA controller is connected only to one device (usually either the memory or the I/O device), and it directly controls data transfer.

Dual-Ended DMA: The DMA controller is connected to both the source and the destination, typically memory and an I/O device.

Arbitrated-Ended DMA: In systems with multiple DMA devices or masters, arbitration is needed to decide which device gets control of the bus. It is more advanced than Dual-Ended DMA.

Interleaved DMA: Interleaved DMA are those DMA that read from one memory address and write from another memory address.

DMA controller

Working of DMA Controller

The DMA controller registers have three registers as follows.

Address register: It contains the address to specify the desired location in memory.

Word count register: It contains the number of words to be transferred.

Control register: It specifies the transfer mode.

All registers in the DMA appear to the CPU as I/O interface registers. Therefore, the CPU can both read and write into the DMA registers under program control via the data bus.

The figure below shows the block diagram of the DMA controller. The unit communicates with the CPU through the data bus and control lines. Through the use of the address bus and allowing the DMA and RS register to select inputs, the register within the DMA is chosen by the CPU.

RD and WR are two-way inputs. When BG (bus grant) input is 0, the CPU can communicate with DMA registers. When BG (bus grant) input is 1, the CPU has relinquished the buses and DMA can communicate directly with the memory.

Buses Device subsystem

- A buses device subsystem isn't a standard term; rather, computer buses are communication pathways that connect various devices and subsystems within a computer, allowing them to share data and signals.
- Key components of a system bus are the **data bus, address bus, and control bus**, which work together to enable efficient communication between the CPU, memory, and I/O devices.
- Examples of devices that utilize buses include storage devices (via a storage bus), graphics cards (on the PCI bus), and peripherals (connected via USB).

External storage system

- External storage refers to devices or media that are used to store data outside of a computer or other digital devices.
- It provides additional space for storing files, documents, media, and other types of data.

- Common Types of External Storage

Hard Disk Drives (HDDs), Magnetic storage with large capacity, Slower than SSDs but cost-effective
Solid-State Drives (SSDs), Flash-based storage with faster read/write speeds, More durable and energy-efficient than HDDs

- **USB Flash Drives Portable**, plug-and-play devices, Ideal for quick file transfers
- **Optical Media (CDs/DVDs),** Laser-based storage, Mostly used for media distribution and archival
- **SD Cards,** Common in cameras and mobile devices, Compact and easy to swap
- **Network Attached Storage (NAS),** External storage connected via a network, Supports multiple users and remote access
- **Cloud Storage,** Data stored on remote servers accessed via the internet, Scalable and accessible from anywhere

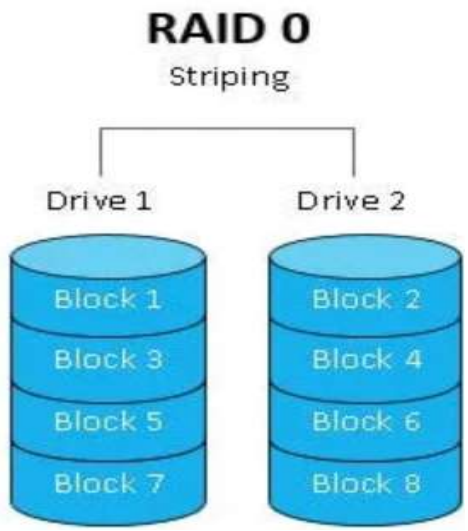
RAID (Redundant Array of Independent Disks) architecture

- RAID (Redundant Array of Independent Disks) is a storage technology that combines multiple physical disk drives into a single logical unit to improve data redundancy, performance, or both.
- There are several different RAID levels, each with its own characteristics and use cases.

- **Different RAID Levels**
- RAID-0 (Stripping)
- RAID-1 (Mirroring)
- RAID-5 (Block-Level Stripping with Distributed Parity)
- RAID-6 (Block-Level Stripping with two Parity Bits)

RAID (Redundant Array of Independent Disks) architecture

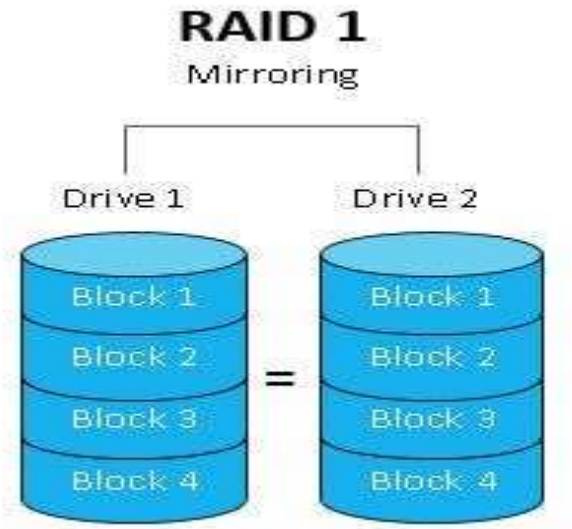
- **RAID-0 (Stripping)**
- RAID 0, often referred to as “striping,” is a basic RAID configuration. In RAID 0, data is divided into blocks and distributed across multiple hard drives or SSDs, which allows for improved read and write performance as multiple drives can be accessed simultaneously. However, RAID 0 lacks data redundancy or fault tolerance. Here are the key characteristics of RAID 0:
- **Data Striping**: Data is split into blocks or stripes, with each block written to a different drive in the RAID 0 array.



RAID (Redundant Array of Independent Disks) architecture

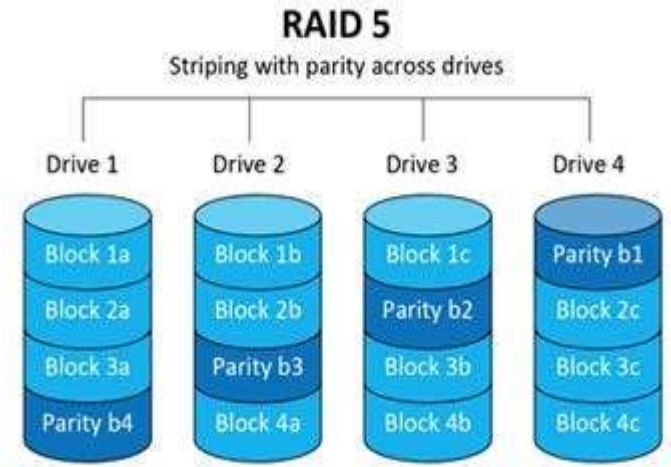
- **RAID-1 (Mirroring)**
- RAID 1 (Mirroring) is a RAID configuration that prioritizes data redundancy and fault tolerance. In RAID 1, data is duplicated or mirrored across two or more drives in the array. This means that every piece of data is simultaneously written to at least two drives, creating identical copies. Key characteristics of RAID 1 include:

- **Data Mirroring**: Data is duplicated across multiple drives, ensuring that each drive contains an identical copy of the data.



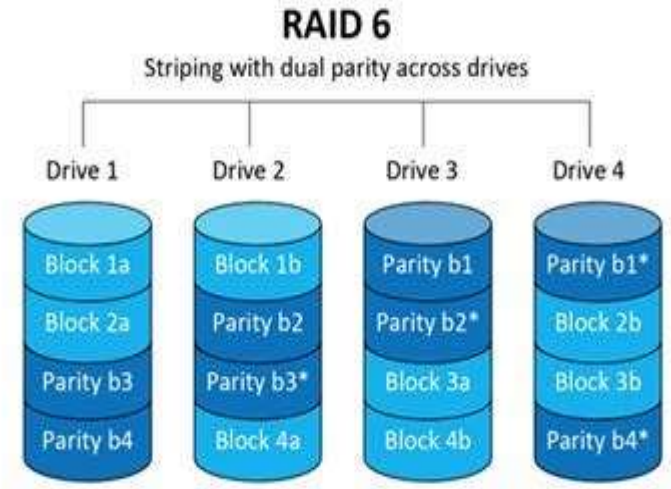
RAID (Redundant Array of Independent Disks) architecture

- **RAID-5 (Block-Level Stripping with Distributed Parity)**
- RAID 5 is a configuration that combines data striping with distributed parity for both performance enhancement and data redundancy.
- RAID 5 requires a minimum of three drives and combines data striping, similar to RAID 0, with distributed parity. In the event of a single drive failure, data can be reconstructed using the parity information distributed across the remaining drives, with no downtime. While read speed is fast, write speed can be somewhat slower due to parity calculations. This RAID level is well-suited for file and application servers with a limited number of data drives.



RAID (Redundant Array of Independent Disks) architecture

- **RAID-6 (Block-Level Stripping with two Parity Bits)**
- RAID 6 is a RAID configuration that builds upon the principles of RAID 5 but offers higher fault tolerance by using dual distributed parity. In RAID 6, like RAID 5, data is divided into blocks and striped across multiple drives. However, it differs in that it uses two separate parity blocks to protect against two drive failures simultaneously, whereas RAID 5 can only tolerate one drive failure.
- **Dual Distributed Parity:** RAID 6 uses two separate parity blocks, each distributed across all the drives in the array. This redundancy allows it to withstand the failure of two drives concurrently without data loss.



RAID (Redundant Array of Independent Disks) architecture

- **Types of RAID Controller**
- There are three types of RAID controller:
- **1. Hardware-Based:**
- Uses a dedicated physical controller to manage hard drives.
- Offers high speed and reliability
- Can work independently from the computer's processor
- Often built into the motherboard or as a separate card
- Think of it as a captain managing the drives smoothly.
- **2. Software-Based:**
- Uses the computer’s processor and memory to manage RAID.
- No special hardware needed
- Cost-effective, but may reduce overall system performance
- Slower than hardware RAID
- Acts like a helpful assistant, but shares the load with other tasks.
- **3. Firmware-Based (Fake RAID):**
- Built into the computer's BIOS/firmware and works during boot-up.
- Needs a driver after the OS loads
- Cheaper than hardware RAID, but still uses CPU resources
- Also known as hybrid RAID or fake RAID
- A startup helper that hands over the job to software once the system runs.